

SSC0501 - Introdução à Ciência de Computação I

Bem Vindos!

SSC0501 - Introdução à Ciência de Computação I

Prof. Matheus Machado dos Santos

Email: matheus.m.santos@icmc.usp.br

Atendimentos: terças-feiras das 17h às 19h.

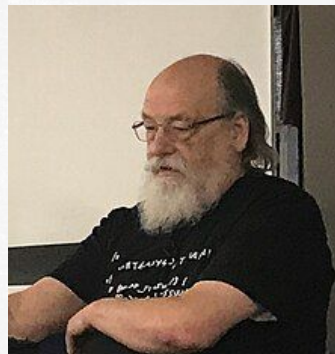
Agendamento de atendimento por e-mail ou google calendar ([link](#))

Linguagem C

- Origem de C está associada ao sistema Unix
- Histórico:
 - 1970: **Ken Thompson** desenvolve B, baseada em **BCPL**, para o primeiro Unix no **DEC PDP-7**
 - 1972: **Dennis Ritchie** projeta a **linguagem C**, baseada na **linguagem B**.
 - 1988: o *American National Standard Institute* (ANSI) define o **padrão ANSI C**



Dennis Ritchie



Thompson, 2019



DEC PDP-7
~16 KB memória

Linguagem C

- É considerada uma **linguagem de nível médio** (combina elementos de baixo nível “assembly” e alto nível if,else,for,while, funções...)
- É “case sensitive”, ou seja, interpreta como diferentes letras maiúsculas de minúsculas.
- É uma linguagem fortemente tipada.

VARIÁVEIS E TIPOS DE DADOS

Variáveis

- Como **armazenar os dados** de entrada, fornecidos pelo usuário?
- O que fazer com os resultados das operações?
- Variáveis são elementos que estão associados a **posições de memória**, cujo objetivo é o armazenamento informações.
- ...por tempo suficiente ao seu processamento

Identificadores

Os identificadores em C devem seguir algumas **regras sintáticas** estabelecidas pela linguagem:

1- Caracteres permitidos:

- Um identificador deve **começar com uma letra (A-Z ou a-z) ou um underscore** (_).
- Os caracteres subsequentes podem incluir letras, dígitos (0-9) e underscores (_).

2- Sensibilidade a maiúsculas e minúsculas:

- C diferencia maiúsculas de minúsculas, ou seja, **variavel**, **Variavel** e **VARIAVEL** são identificadores distintos.

Identificadores

3- Não pode ser uma palavra-chave:

- Identificadores não podem ser nomes reservados da linguagem, como int, return, for, etc.

4- Comprimento:

- O padrão C99 exige que ao menos os primeiros 63 caracteres de um identificador sejam considerados únicos para nomes de variáveis e funções, e os primeiros 31 caracteres para macros.

Palavras-chave de C (ANSI)

auto break case char const continue
default do double else enum extern float
for goto if int long register return short
signed sizeof static struct switch typedef
union unsigned void volatile while

Palavras-chave de C (ANSI)

auto break case **char** **const** continue
default do **double** else enum extern **float**
for goto if **int** **long** register return **short**
signed sizeof **static** **struct** switch typeof
union **unsigned** void **volatile** while

Tipos de Identificadores

1- Variáveis

```
int idade;  
float temperatura;  
char letra;
```

2- Funções:

```
int calcular_soma(int a, int b);  
void imprimirMensagem();
```

Tipos de Identificadores

1- Variáveis

```
int idade;  
float temperatura;  
char letra;
```

Identificadores!

2- Funções:

```
int calcular_soma(int a, int b);  
void imprimirMensagem();
```

Tipos de Identificadores

3- Macros (via #define):

```
#define TAMANHO_MAXIMO 100
```

4- Tipos definidos pelo usuário (typedef e struct):

```
typedef unsigned int Idade;  
struct Pessoa {  
    char nome[50];  
    int idade;  
};
```

Tipos de Identificadores

3- Macros (via #define):

```
#define TAMANHO_MAXIMO 100
```

Identificadores!

4- Tipos definidos pelo usuário (typedef e struct):

```
typedef unsigned int Idade;  
struct Pessoa {  
    char nome[50];  
    int idade;  
};
```

Boas Práticas para Identificadores

Existem algumas convenções que podem ser adotadas:

1- Escolha nomes significativos (Se você não estiver em uma competição de programação):

```
int x;    // Ruim
int contador_de_clientes; // Melhor
```

2- Use convenções de nomenclatura:

- **Camel case:** calcularSoma(), numeroClientes (comum em funções e variáveis)
- **Snake case:** calcular_soma(), numero_clientes (comum em macros e nomes de arquivos)
- **Uppercase para macros:** #define TAMANHO_MAXIMO 100

Boas Práticas para Identificadores

Existem algumas convenções que podem ser adotadas:

3- Evite underscores no início do nome:

- Identificadores começando com `_` seguidos de uma letra maiúscula (`_Exemplo`) ou com dois underscores (`__exemplo`) são reservados para a implementação do compilador.

4- Diferencie nomes de variáveis e funções de forma intuitiva:

```
int tamanho_lista; // Variável  
int obter_tamanho_lista(); // Função
```

Tipos de Dados

- O tipo de uma variável define os valores que ela pode assumir e as operações que podem ser realizadas com ela.
- Descreve a natureza da informação

Ex:

- variáveis tipo **int** recebem apenas valores **inteiros**
- variáveis tipo **float** armazenam apenas valores **reais**

Tipos Primitivos

Os tipos primitivos são aqueles diretamente suportados pela linguagem:

- **Inteiros (int):** Armazenam números inteiros. O tamanho pode variar dependendo do compilador e da arquitetura, mas geralmente ocupa **4 bytes**.
- **Caractere (char):** Representa um único caractere e geralmente ocupa **1 byte**.
- **Ponto flutuante (float e double):** Armazenam números reais. O float normalmente ocupa **4 bytes** e tem menor precisão, enquanto double ocupa **8 bytes** e possui maior precisão.
- **Void (void):** Representa ausência de tipo e é frequentemente usado em funções que não retornam valor.

Modificadores de Tipos

- Os modificadores alteram algumas características dos tipos básicos para adequá-los a necessidades específicas.

Modificadores:

- **signed:** indica número com sinal (inteiros e caracteres)
- **unsigned:** número apenas positivo (inteiros e caracteres)
- **long e long long:** aumenta abrangência (inteiros e reais)
- **short:** reduz a abrangência (inteiros)

Abrangência dos dados

(Depende da arquitetura 16, 32, 64 bits)

Tipo	Tamanho (bytes)	Intervalo (Signed)	Intervalo (Unsigned)
char	1	-128 a 127	0 a 255
short	2	-32.768 a 32.767	0 a 65.535
int	4	-2.147.483.648 a 2.147.483.647	0 a 4.294.967.295
long	4 ou 8	Depende da plataforma	Depende da plataforma
long long	8	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807	0 a 18.446.744.073.709.551.615
float	4	$\sim \pm 3.4 \times 10^{-38}$ a $\pm 3.4 \times 10^{38}$	N/D
double	8	$\sim \pm 1.7 \times 10^{-308}$ a $\pm 1.7 \times 10^{308}$	N/D

Qual seria a abrangência de um int de 3 bits?

$2^3 = 8$, logo, abrange de 0 a 7 (8 números contando com o zero)

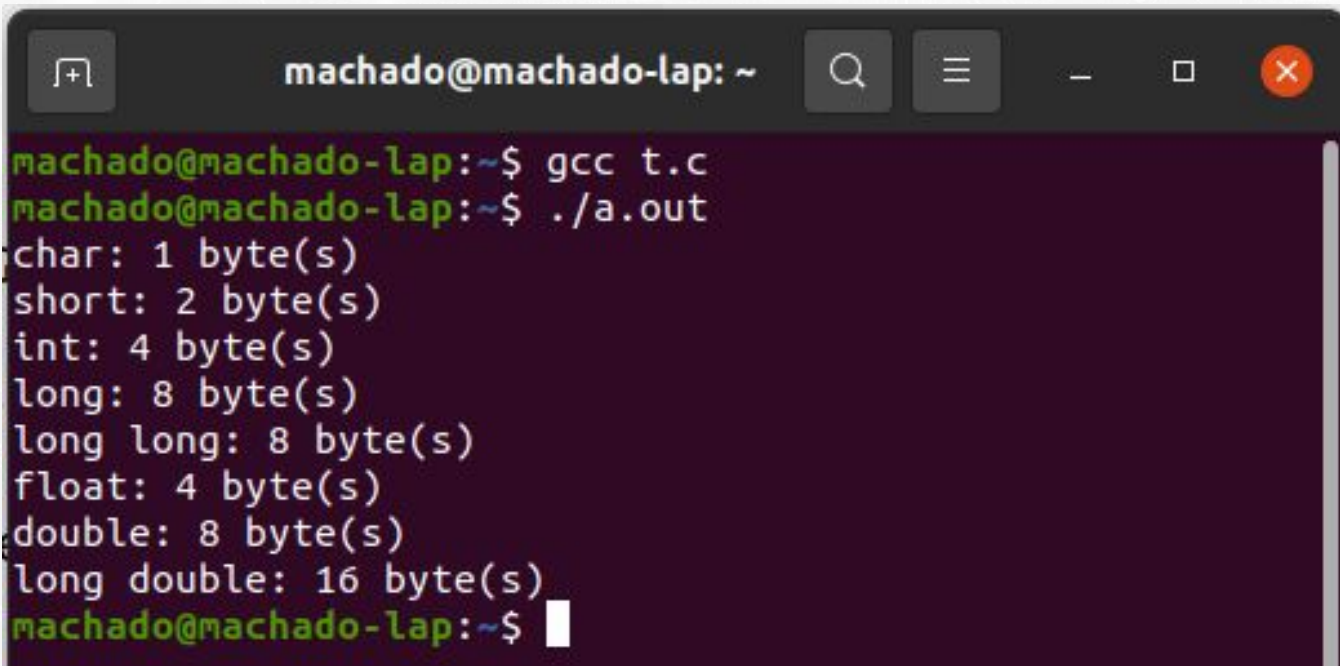
Decimal	Binário
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Código prático para saber o tamanho das variáveis

```
#include <stdio.h>
```

```
int main() {  
    printf("char: %lu byte(s)\n", sizeof(char));  
    printf("short: %lu byte(s)\n", sizeof(short));  
    printf("int: %lu byte(s)\n", sizeof(int));  
    printf("long: %lu byte(s)\n", sizeof(long));  
    printf("long long: %lu byte(s)\n", sizeof(long long));  
    printf("float: %lu byte(s)\n", sizeof(float));  
    printf("double: %lu byte(s)\n", sizeof(double));  
    printf("long double: %lu byte(s)\n", sizeof(long double));  
  
    return 0;  
}
```

Código prático para saber o tamanho das variáveis

A terminal window with a dark purple background and white text. The window title bar shows 'machado@machado-lap: ~' and standard window controls. The terminal shows the execution of a C program that prints the size of various data types in bytes.

```
machado@machado-lap: ~$ gcc t.c
machado@machado-lap: ~$ ./a.out
char: 1 byte(s)
short: 2 byte(s)
int: 4 byte(s)
long: 8 byte(s)
long long: 8 byte(s)
float: 4 byte(s)
double: 8 byte(s)
long double: 16 byte(s)
machado@machado-lap: ~$
```

Declaração de variáveis

- A declaração de uma variável segue o modelo:

TIPO_VARIÁVEL lista_de_variaveis;

Ex:

```
int x, y, z;
float f;
unsigned int u;
long double df;
char c = 'A';           /* variavel definida e iniciada */
char s[ ] = "vetor de caracteres";
```

OPERADORES

Operadores

- Correspondem a **símbolos simples ou combinados** que representam **operações** de natureza: **aritmética, relacional ou lógica**.
- Podem ser classificados também quanto a quantidade de elementos (operandos) sob os quais incidem, i.e., unários, binários ou ternários

Operadores Aritméticos

- Representam as operações aritméticas básicas

Operação	Operador
Adição	+
Subtração	-
Multiplicação	*
Divisão	/
Resto	%
Incremento	++
Decremento	--

Operadores Relacionais

- Estabelecem relações/comparações

Operação	Operador
Igualdade	==
Diferença	!=
Maior	>
Maior ou igual	>=
Menor	<
Menor ou igual	<=

Operadores Lógicos

- Representam as operações básica dada na lógica matemática

Operação	Operador
Negação	!
Conjunção	&&
Disjunção	

Operadores bit a bit (existe também...)

- Representam operações bit a bit sobre os dados

Operação	Operador
AND (e)	&
OR (ou)	
XOR (ou exclusivo)	^
NOT (complemento)	~
Deslocamento a direita	>>
Deslocamento a esquerda	<<

Operador de Atribuição

- Forma geral:
variável = expressão ou constante
- Armazena o conteúdo dado a direita no elemento dado à esquerda
- Em C é permitido múltiplas atribuições em um mesmo comando:

```
x = y = z = 0;
```

Expressões

Expressões são compostas por:

- Operandos: a, b, x, Meu_dado, 2, ...
- Operadores: +, -, %, ...
- Pontuação: ()
- Funções: sin(), abs(), sqrt(), ...

Ex:

X

14

$x + y$

$(x + y) * z + w - v$

$(-b + \text{sqrt}(\text{delta})) / 2 * a$

Expressões

- Expressões retornam um valor:

```
x = 5 + 4 /* retorna 9 */
```

Esta expressão atribui 9 a x e retorna 9 como resultado da expressão (sim atribuição também retorna o resultado da atribuição, o que permite realizar múltiplas atribuições em linha): `x = y = z = k = 5;`

```
((x = 5 + 4) == 9) /* retorna verdade = 1 */
```

Na expressão acima, além de atribuir 9 a x, o valor retornado é utilizado em uma comparação

Expressões

- A ordem em que uma expressão é avaliada depende da prioridade dos operadores e da pontuação.
- Expressões podem aparecer em diversos pontos de um programa

comandos

```
/* x = y; */
```

parâmetros de funções

```
/* sqrt(x + y); */
```

condições de teste

```
/* if (x == y) */
```

ESTRUTURA BÁSICA

Programação em C

- Todo programa, escrito na linguagem C, deve apresentar uma função principal chamada main, que define todo o corpo do programa

- Exemplo:

```
int main() {  
    /* corpo do  
programa */  
}
```

Comandos de Saída

- Empregados para que o sistema forneça, em um dispositivo de saída, as mensagens e resultados de seu processamento.
- O dispositivo padrão de saída é o monitor.
- A linguagem C oferece alguns comandos de saída, mas o que apresenta propósito mais geral é o printf.

Comando PRINTF()

- Sintaxe:

`printf("Mensagem", lista de variáveis);`

- Funcionamento:

- O comando escreve a mensagem dada no dispositivo padrão de saída, realizando a substituição das máscaras de formatação encontradas pelas respectivas variáveis dadas na lista subsequente a

Máscaras de formatação

- Símbolo de por cento seguido de uma letra:
 - %c Caractere
 - %d Inteiros com sinal
 - %u Inteiros sem sinal
 - %f Números reais
 - %e Notação científica
 - %x Números em hexadecimal
 - %s Cadeia de caracteres (strings)

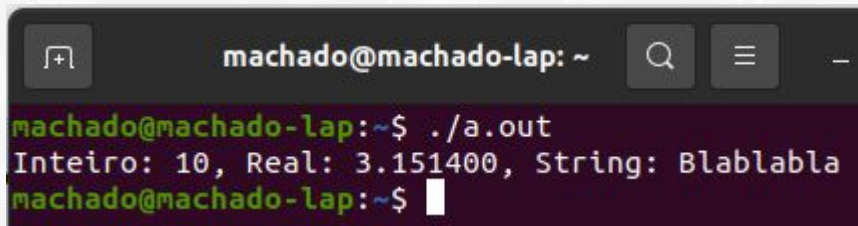
Exemplo

- Saída formatada **printf()**. Ex:

O trecho abaixo:

```
int i = 10;
float r = 3.1514;
char s[] = "Blablabla"; /* cadeia de caracteres */
printf("Inteiro: %d, Real: %f, String: %s\n", i, r, s);
```

Produz: Inteiro: 10, Real: 3.151400, String: Blablabla

A terminal window with a dark background. The prompt is 'machado@machado-lap: ~'. The user enters './a.out' and the program outputs 'Inteiro: 10, Real: 3.151400, String: Blablabla'. The prompt returns to 'machado@machado-lap: ~'.

```
machado@machado-lap: ~  
machado@machado-lap:~$ ./a.out  
Inteiro: 10, Real: 3.151400, String: Blablabla  
machado@machado-lap:~$
```

Constantes do Tipo Char

- Barra invertida seguido de um caractere:

\a bip

\b backspace

\n nova linha

\r return

\t tabulação horizontal

\' apóstrofe

\” aspas

\\ barra invertida

Comandos de entrada

- Utilizado para receber dados fornecidos pelo usuário (dados de entrada) e armazená-los na memória principal (em variáveis)
- Os dados são fornecidos ao sistema por meio de um dispositivo de entrada, cuja configuração dada como padrão é o teclado.
- A linguagem C oferece vários comandos de entrada, cada qual mais indicado para uma situação em particular.
- O principal comando de entrada é o scanf

Comando SCANF()

- Sintaxe:

```
scanf("formato", &variável);
```

- Funcionamento:

- O comando coleta as informações dadas no dispositivo padrão de entrada, interpretando as informações segundo a máscara de formatação e armazenando na(s) respectiva(s) variável(is) dada(s) subsequentemente ao formato.
- O dispositivo padrão é dado pela variável stdin

Exemplo

- Entrada formatada **scanf()**.
 - Ex:
int i; float r; char str[10];
scanf("%d",&i);
scanf("%f",&r);
scanf("%s",&str);
 - Ou ainda:
int dia, mes, ano;
scanf("%d/%d/%d", &dia, &mes, &ano);

Exemplo Completo com Entrada e Saída

```
#include <stdio.h>
```

```
int main(){  
    int num1, num2, media;  
    printf("Digite 2 numeros: ");  
    scanf("%d %d",&num1, &num2);  
    media=(num1+num2)/2;  
    printf("\nMedia: %d", media);  
    getch();  
    return 0;  
}
```

Prática!

Código Matricula: R4SM

<https://runcodes.icmc.usp.br/offerings/view/83>

[run.codes]

Menu Professor

matheus.m.santos@icmc.usp.br

Hora do Servidor: 14/03/2025 18:46:06

Home > SSC0501

SSC0501 - Introdução à Ciência de Computação I

Professores/Monitores

Professores: Matheus Machado dos Santos
Turma: 2025101
Universidade: USP
Ativa até: 21/07/2025



Código de Matrícula

R4SM



Novo Exercício



Enviar E-mail



Ver Notas



Exportar Tabela de Notas

Exercícios

No.	Exercício	Status	Casos Corretos	Nota	Entregas	Participantes	Prazo de Entrega	Ações
1	Hello World	Finalizado	1/1	10.00	78	43/46	14/03/2025 21:00:00	Ver Detalhes Remove Exercício
2	1.01 Maior número	Não Entregue	0/6	0	0	0/46	19/03/2025 23:59:59	Ver Detalhes Remove Exercício
3	1.02 Par ou ímpar	Não Entregue	0/7	0	0	0/46	19/03/2025 23:59:59	Ver Detalhes Remove Exercício
4	1.03 Positivo, negativo ou zero	Não Entregue	0/7	0	0	0/46	19/03/2025 23:59:59	Ver Detalhes Remove Exercício
5	1.04 Maior de três números	Não Entregue	0/7	0	0	0/46	19/03/2025 23:59:59	Ver Detalhes Remove Exercício

SSC0501 - Introdução à Ciência de Computação I

Obrigado pela atenção!!